



FINAL CURRICULUM PROJECT

Mini-SIMS Portal

A compact Student Information Management System built with React, TypeScript, and Tailwind CSS — engineered with a focus on understanding every layer of the stack.



React 18 • TypeScript • Tailwind CSS v4 • Vite

Project Vision & Core Purpose



Deliberate Learning

Built as a final curriculum capstone project with a strict rule: prioritize complete comprehension of React fundamentals over pasting prefabricated solutions. Every state handler, hook, and route transition is coded intentionally.



Compact SIS Solution

Consolidates daily academic administrative tasks—overview metrics, dynamic student attendance tracking, fee management, and PDF compilation—into a responsive, zero-bloat web application interface.

Key Application Features



Interactive Dashboard

Displays stat cards covering total fees due, attendance rates, next scheduled class details, and system alerts.



Attendance Tracker

Toggle present/absent states with a live "X of Y present" counter, integrated directly with a mock REST API.



Fees & PDF Receipts

Track paid vs. unpaid statuses, add custom fees, display Recharts breakdown, and generate client-side PDF receipts via jsPDF.



Multi-Page Routing

Client-side navigation powered by React Router, paired with a responsive collapsible layout for mobile screens.

Technology Stack



React 18 + TypeScript (Vite): Fast build setup with full static typing and modular component architecture.



Tailwind CSS v4 & shadcn/ui: Modern utility-first styling combined with accessible UI primitives built on Base UI.



Recharts Data Visualization: Interactive bar charts rendering paid vs. unpaid fee metrics visually.



jsPDF Document Engine: Generates formatted tuition fee receipts on demand directly inside the client browser.



json-server Mock REST API: Simulates a persistent backend database (`db.json`) running on port 3001.

Project Structure & Architecture

Directory Breakdown

src/components/ — Shared domain components
(Attendanceltem, Feeltem, DashboardCard)

src/components/ui/ — Primitive shadcn/ui building blocks
(Card, Button, Badge, Input)

src/pages/ — Route-level pages (Dashboard, Attendance, Fees)

src/lib/ — Utility helpers including custom jsPDF receipt
compilers

Core Entry & Data Persistence

App.tsx: Manages top-level application state, native `fetch` data
lifecycles, and client routing definitions.

db.json: Serves as the central JSON database consumed by
`json-server` to persist student profiles, logs, and fee records
locally.

Getting Started & Execution

The application relies on concurrent execution of both the Vite client server and the mock API server:

```
# Terminal 1: Install dependencies & run Vite frontend (Port 5173)
```

```
npm install
```

```
npm run dev
```

```
# Terminal 2: Run local json-server mock API (Port 3001)
```

```
npm run server
```

API Dependency Note

Both processes must run simultaneously — the React frontend fetches all live student attendance and tuition data directly from `http://localhost:3001` via `json-server`.

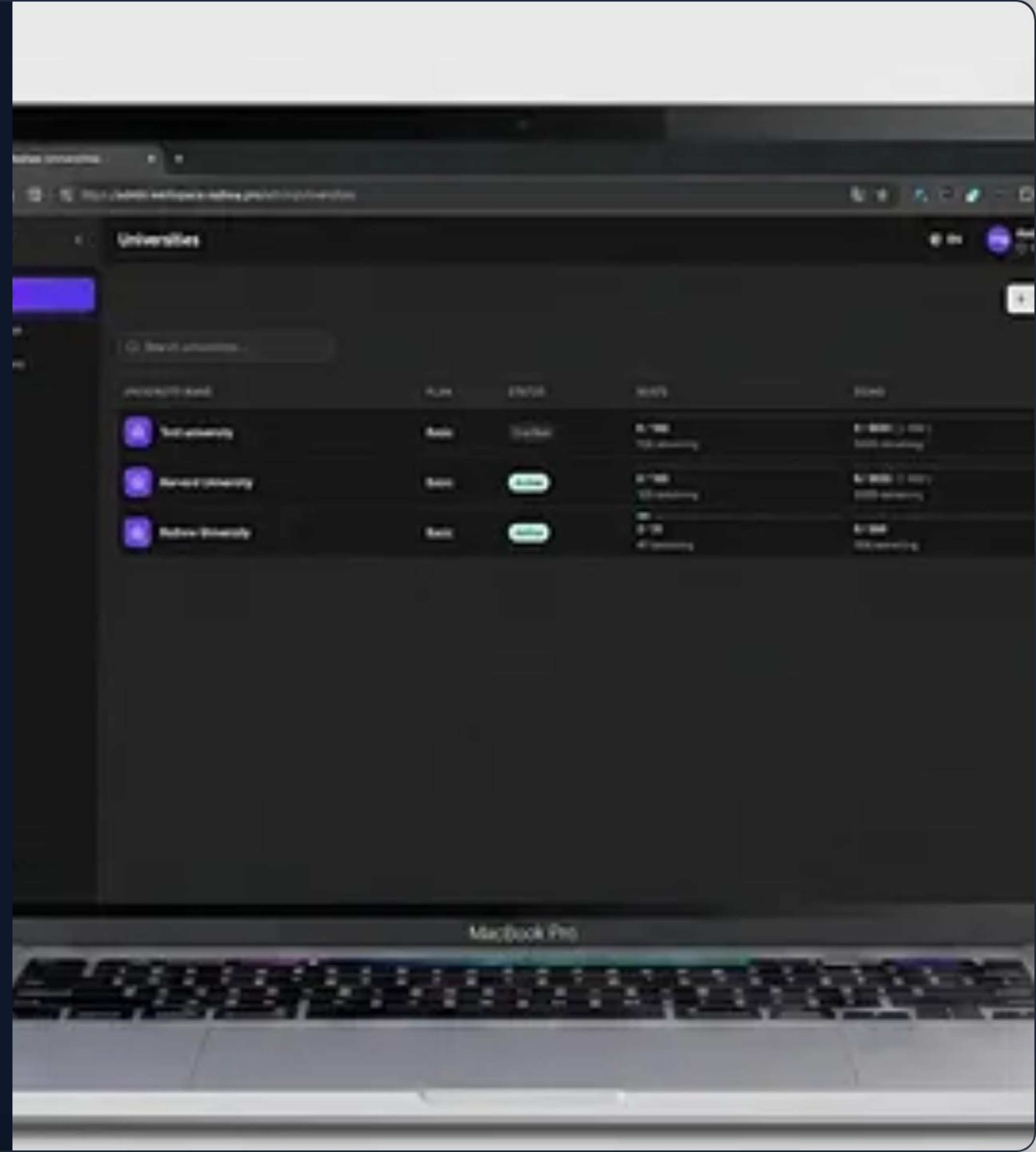
UI & Design System

Custom Dark Palette

Engineered using Slate color tones to reduce visual eye strain during administrative sessions, anchored by high-contrast cyan accents (#38bdf8).

Component Consistency

Leverages shadcn/ui components (Card, Badge, Button) styled via Tailwind CSS v4, providing accessible and responsive UI across tablet and mobile displays.



Scope Notes & Engineering Trade-Offs



State Management Choice

Opted for React's native `useState` and `useEffect` hooks over Redux to deepen foundational understanding of state lifting and prop drilling dynamics.



Native Fetch API

Utilized the native browser `fetch` API instead of third-party HTTP libraries like Axios to understand raw Request/Response lifecycle handling.



Pruned Stretch Features

Intentionally cut timetable modules, role-based routing, and AI risk scoring from initial scope to prioritize deep quality and bulletproof execution of core features.

Core Competencies & Mastery

✔ **Lifting State Up:** Synchronizing shared student data seamlessly across Dashboard overview cards and Attendance/Fees sub-views.

↻ **Effect Lifecycles:** Managing asynchronous mock API requests, cleanup handlers, and side-effects clean of memory leaks.

🗄 **CRUD Operations:** Implementing functional state updates to create, read, and update student fee logs dynamically.

📦 **Component Architecture:** Structuring reusable atomic UI primitives alongside page-level route container views.

Live Deployment & Resources

100%

Stack Understanding Focus

0

Copy-Pasted Abstractions



Live Project Deployment

<https://mini-sims-portal-53vp.vercel.app/>



Curated Color Palette

<https://coolors.co/08090c-d6dad9-0c0b0e-09090b-595d60>

Thank You!

Questions, technical discussions, or suggestions are welcome.



mini-sims-portal-53vp.vercel.app



React + TypeScript Capstone

Image Sources



<https://cdn.dribbble.com/userupload/48310864/file/37e94c1e7318fbefcc00e67169806043.jpg?crop=506x75-3199x2095&format=webp&resize=400x300&vertical=center>

Source: dribbble.com